



BREH

Behavior Regulation Environment Handler

*Sustainable agentic AI scaling platform
through LLM-driven DAG orchestration*

INDUSTRY

\$600B+

global cloud market;
orchestration spend
growing >20% YoY

PROBLEM

Manual

scaling causes
outages & cost overruns
humans can't watch 24/7

WHY AI

LLM + ML

reason over signals,
correlate metrics,
act autonomously



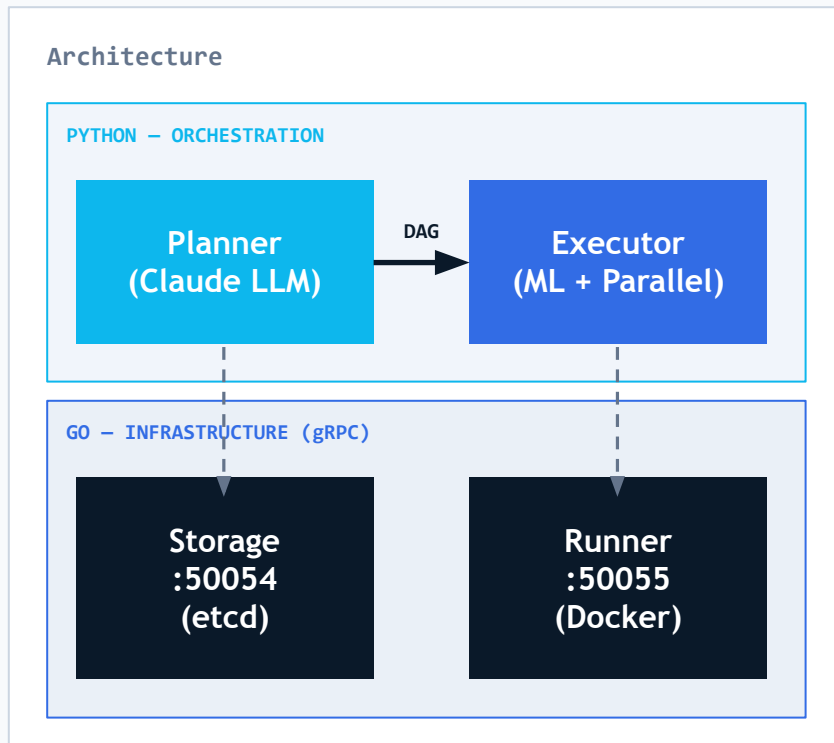
System Overview

BREH // Industry Defense

One LLM call emits a DAG // Executor walks it in parallel // ML routes each step

What BREH Does

- 1 Receives operational alerts (“high latency detected”)
- 2 Plans a multi-step response as a DAG — one LLM call
- 3 Executes steps in parallel with ML-driven routing
- 4 Decides: scale up, maintain, or investigate further
- 5 Logs every decision to an append-only audit trail



Thesis: one LLM round-trip is enough, the executor handles the rest with frontier-based dispatch.



Three Capstones, One System

BREH // Industry Defense

each prior project supplied a missing capability

PROJECT 3

ML Scaling

Gradient Boost classifier on Alibaba
2026-GenAI traces.

96% test accuracy, 86% minority recall.

→ ML brain for routing decisions

PROJECT 5

LLM Function Calling

Structured-output fine-tuning patterns;
GPT-2 on BFCL.

Single-call JSON DAG contract.

→ Single-call DAG generation

PROJECT 6

Agentic Orchestration

smolagents ToolCallingAgent with Pydantic
validation.

Provides benchmark baseline.

→ Tools + comparison baseline

BREH: the LLM plans WHAT to do; ML decides HOW to execute it

Rationale: P3 alone can predict but not act. P5 alone can plan but not route. P6 alone can act but only sequentially. Together they cover the full loop.



Inside the Models

BREH // Industry Defense

Gradient Boosting routes execution // Transformer plans the DAG

MODULE 3

Gradient Boost Classifier

predicts needs_scale_up from live cluster signals

96%

test accuracy

86%

minority recall

1.7%

train/test gap

Top feature: GPU duty cycle (strongest positive signal)

TRAINING & DEPLOYMENT

- 1 Ingest Alibaba 2026-GenAI cluster traces
- 2 Engineer features (CPU duty cycle, QPS, queue RT)
- 3 Train GBoost on needs_scale_up label for future inference
- 4 Deploy as ScalingPredictorTool; live metrics override LLM args

MODULE 5

Transformer (GPT-2 fine-tune)

emits structured JSON function calls – the DAG contract

Loss: 11.39 → 0.95 over 10 epochs



TRAINING & FINDING

- 1 Load BFCL function-calling dataset
- 2 Format as JSONL pairs (instruction → JSON call)
- 3 Fine-tune GPT-2 on Colab T4 across 10 epochs
- 4 In-domain near-perfect; out-of-domain failure = key finding

In BREH: the Transformer plans WHAT (DAG); Gradient Boost decides HOW (scale or hold) using live metrics, not LLM-generated args.



The Data Contract

BREH // Industry Defense

Graph & Step // pydantic-validated, flows through every edge of the system

MODULE 6

Graph

one LLM call emits a full Graph

```
id: str
request: str
created_at: int
steps: list[Step]
```

Role: the unit of planning. Persisted to etcd via Storage; consumed whole by the Executor.

Step

depends_on encodes the DAG; weight gates execution

```
id: str
tool: str
args: dict
weight: float
depends_on: list[str]
```

Role: one unit of work. tool + args dispatch; weight ≥ 0.85 to run; depends_on resolves frontier.

WHY IT MATTERS // Graph 1—* Step. Pydantic rejects malformed plans before any execution; the same schema serializes to etcd and into spawned containers.



Technical Decisions & Tradeoffs

BREH // Industry Defense

every choice had a cost – these are the ones I owned

DECISION

WHY

ALTERNATIVE REJECTED

DAG + parallel execution

wall-clock matters for real-time scaling

Sequential is simpler but 2–3× slower

Python + Go split

each language plays to its strengths

Go lacks direct ML-integration

Live metrics override LLM args

LLMs hallucinate plausible-looking metrics

trust LLM-generated data == wrong decisions

Weight threshold (0.85)

skip low-confidence steps to bound cost

execute everything is risky and expensive

Container isolation

safety for executing untrusted LLM code

all inline is a security risk

HARDEST TRADEOFF // DAG complexity vs. execution speed. Chose speed with safeguards: cycle detection, bounded iterations (max 3), weight thresholds.



Ethical Considerations

BREH // Industry Defense

autonomy + cost + opacity = real risk surface

RISKS // what could go wrong

Autonomous action

system spawns containers without human approval

Cost amplification

incorrect scaling could incur significant cloud bills

Training data bias

Gradient Boost trained on historical traces may not generalize

Decision opacity

LLM reasoning is not fully interpretable

SAFEGUARDS // what I built in

Weight threshold (0.85)

low-confidence steps are skipped

Append-only audit log

every decision logged to logs/scaling.jsonl

Max iteration bound (3)

prevents runaway execution and cost

Container isolation

read-only FS, dropped caps, resource limits

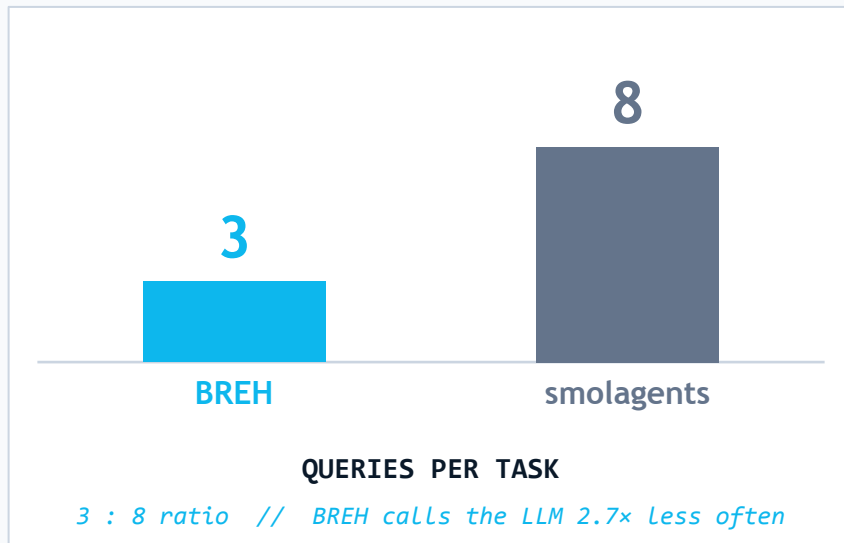
For production: add hard cost caps, human approval gates for high-impact actions, real-time alerting, and a retraining pipeline driven by the audit log.



Evaluation & Results

BREH // Industry Defense

fewer queries, lower spend // BREH vs. smolagents on the same prompts



FUNCTIONAL

End-to-end runs produce results.

PERFORMANCE

Fewer calls + parallel dispatch.

ROUTING

ML decisions track metric patterns.

AUDIT

Full JSONL traceability.

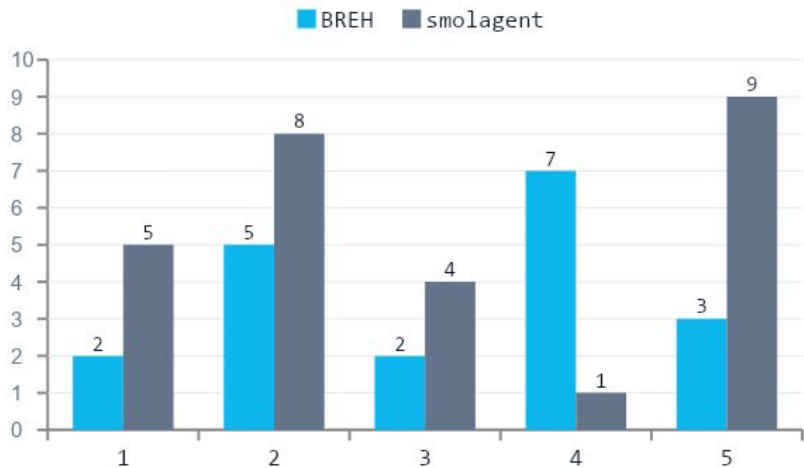
Tested with: "High Latency detected" // "QPS=10 QUEUE_RT=6, should I scale up?" // research-style prompts



Evaluation & Results

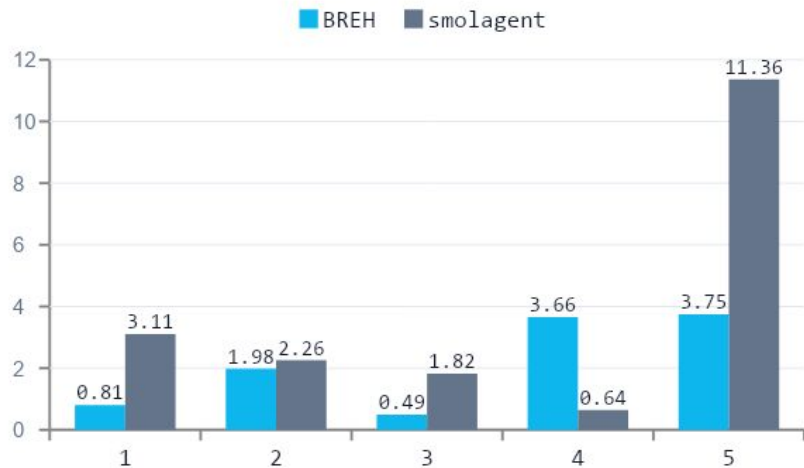
BREH // Industry Defense

real per-task data from results.csv // Claude Haiku 4.5 // 5 prompts, seed 0



QUERIES PER TASK (LLM calls)

19 vs 27 total // BREH calls 1.4× less often



COST PER TASK (cents, USD)

\$0.107 vs \$0.192 total // BREH spends 1.8× less

ACROSS 5 TASKS // BREH used **40K vs 169K** input tokens (**4.2× fewer**). Honest caveat: smolagent wins on math — pure reasoning, no tools needed.

Prompts: "J2EE Spring Boot server" // "QPS=10 QUEUE_RT=6, scale up?" // multihop // hard math // hard biology (MR causality)



Limitations & Risks

BREH // Industry Defense

honest gaps between prototype and production

LIMITATION	IMPACT	PATH FORWARD
Single Docker host	No true distributed scaling	<i>Design supports Kubernetes extension</i>
Lack of XAI	Smolagents thoughts are hidden	<i>Logging inside containers helps retain this</i>
Limited test coverage	Small prompt set tested	<i>Framework supports larger benchmarks</i>
No cost controls	Could overspend if misconfigured	<i>Graph Step caps + alerting in production</i>

FAILURE CASE OBSERVED //

When the LLM can answer directly, as seen in the math_hard smolagent traces, BREH fails to single-trip the answer with the mother LLM. The failure is considered a lack of software engineering and not directly applied to the architecture or ML approaches.



Professional Relevance

BREH // Industry Defense

what this project demonstrates for real-world AI work

TECHNICAL

Full-Stack AI Engineering

ML integration, LLM orchestration, gRPC services, Docker isolation, Python + Go across language boundaries.

ARCHITECTURAL

Systems Design

Tradeoff analysis, language boundary decisions, parallel dispatch, frontier-based scheduling.

ETHICAL

Responsible AI Reasoning

Risk identification, safeguard design, transparency through audit logs, conservative defaults.

COMMUNICATION

Technical Writing

Diagrams (UML class, component, sequence, activity), benchmark reports, explaining complex systems.

NEXT STEPS // Kubernetes integration • online retraining from audit logs • human approval gates • multi-LLM routing for cost optimization



Synthesis Achieved

Three capstone projects unified into a coherent, working system.

ML

data-driven
routing decisions

LLM

flexible, context-aware
planning

SYSTEM

efficient distributed
execution

Questions?

design choices • integration strategy • ethics • limitations • deployment